

GUIÓN 1: DESARROLLO DE CÓDIGO PARA EL PROCESADOR ARM

INTRODUCCIÓN

La asignatura Proyecto Hardware tiene por objetivo desarrollar un proyecto que permita evaluar el uso óptimo y eficiente del procesador y sus periféricos. Para gestionar mejor el proyecto está dividido en varias etapas con entregas parciales. En total habrá 5 guiones, 5 entregas de código, 6 defensas en el laboratorio (5 parciales y la final), 2 entregas de memorias y entregas de resultados intermedios.

Este guion corresponde al trabajo de las tres primeras sesiones de laboratorio, en las que vamos a optimizar el rendimiento de un código desarrollado en C.

A partir de los requisitos fijados en este guion se desarrollará primeramente código en C y se tratará de optimizar desarrollando las mismas funciones en ensamblador ARM, adecuado para optimizar el rendimiento, y Thumb, adecuado para disminuir el tamaño del código. Ejecutando un emulador de un System-on-Chip ARM con el entorno de desarrollo Keil μ Vision, se comprobará el funcionamiento, se medirá el tamaño y el tiempo de ejecución y compararemos el rendimiento obtenido con las distintas versiones, con el que consigue el compilador al activar sus distintas opciones de optimización.

Importante, en todas las entregas de la asignatura, os debéis asegurar de que vuestro código funciona correctamente, para ello, en esta primera entrega desarrollaréis una función que verifique que todas las versiones del código den un resultado equivalente. Además, en cada entrega, os hacéis responsables de la autoría del código y todo el trabajo realizado.

ÍNDICE

INTRODUCCIÓN	1
ÍNDICE	1
OBJETIVOS	2
CONOCIMIENTOS PREVIOS NECESARIOS	2
ENTORNO DE TRABAJO	2
MATERIAL DISPONIBLE	2
ESTRUCTURA DE LA PRÁCTICA	3
PROGRAMA A DESARROLLAR: CAPA NEURONAL EN PUNTO FIJO	3
TAREAS A REALIZAR	6
PUNTOS DE VERIFICACIÓN EN CADA ENTREGA	9
APARTADO OPCIONAL 1:	10
EVALUACIÓN DE LA PRÁCTICA	10
ANEXO 1: RECOMENDACIONES PARA LA REALIZACIÓN DE LA MEMORIA	11
ANEXO 2: ENTREGA DEL CÓDIGO Y DE LA MEMORIA	11
ANEXO 3: ALGUNOS CONSEJOS DE PROGRAMACIÓN EFICIENTE PARA OPTIMIZAR EL CÓDIGO ENSAMBLADOR	12

OBJETIVOS

- Optimizar el rendimiento acelerando las funciones computacionalmente más costosas.
 - Familiarizarse con el entorno de trabajo
 - Comprender la funcionalidad del código suministrado
 - Combinar código en ensamblador con código en C
 - Entender la finalidad y el funcionamiento del ABI (ATPCS)
 - Depurar correctamente el estado arquitectónico (registros y memoria).
 - Optimizar código en distintos lenguajes y verificar resultados.
 - Analizar y medir rendimiento
- Manejar documentación técnica
- Gestionar el tiempo de trabajo del proyecto correctamente.
- Documentar y presentar los resultados. Redactar memorias técnicas

CONOCIMIENTOS PREVIOS NECESARIOS

En esta asignatura cada estudiante necesitará aplicar contenidos básicos de arquitectura y tecnología de computadores adquiridos en asignaturas previas, en particular, Arquitectura y Organización de Computadores I y II.

ENTORNO DE TRABAJO

Esta práctica se realiza con **Keil μ Vision**, este entorno es capaz de editar, depurar, compilar código en C y ensamblador, además de analizar el rendimiento de los binarios generados. En prácticas sucesivas veremos como también es capaz de emular diferentes periféricos y entrada salida del microcontrolador.

Trabajaremos sobre el microcontrolador LPC2105. Se utilizará la versión 5. En Moodle de la asignatura tenéis información del proceso de instalación y fuentes.

MATERIAL DISPONIBLE

Debéis utilizar el siguiente material para prácticas (disponible en Moodle):

- Manuales de Keil μ Vision
- Documentación del sistema emulado microprocesador LPC2105
- Manuales de la arquitectura de referencia: C, ARM, AAPCS,...
- Directrices para redactar una memoria técnica, imprescindible para redactar la memoria de la práctica. Es donde se define la estructura de la memoria a entregar.
- Wikis con información técnica adicional
- Proyecto para Keil μ Vision con los códigos fuentes, este documento y las diapositivas de las clases de teoría.

ESTRUCTURA DE LA PRÁCTICA

La práctica completa se debe realizar en **3 sesiones**. En la siguiente sesión tendrá lugar la entrega presencial del trabajo y una semana después se entregará la memoria a través de Moodle.

Antes de la **primera sesión** es imprescindible haberse leído este guion y conocer la documentación suministrada. Para entrar en la **segunda sesión** de prácticas es obligatorio traer el código fuente mínimo hasta el paso 4, si es posible el del paso 5 o incluso más avanzado. La **tercera sesión** se debería dedicar a realizar las métricas de los apartados 7 y 8, por lo que se deberá mostrar al inicio de la sesión el correcto funcionamiento del código hasta el punto 6 incluido.

PROGRAMA A DESARROLLAR: CAPA NEURONAL EN PUNTO FIJO

Debéis desarrollar y depurar, en C (primero) y en ensamblador ARM y Thumb (consecutivamente), vuestro código que implemente una pequeña capa neuronal completamente conectada usando aritmética entera en punto fijo. Este tipo de operación aparece en redes neuronales y en otros algoritmos de procesamiento de datos, y consiste principalmente en realizar productos escalares entre un vector de entrada y un conjunto de pesos.

Una capa completamente conectada calcula un vector de salida a partir de un vector de entrada. Cada elemento de salida se obtiene mediante una neurona. Cada neurona multiplica todos los valores de entrada por sus pesos correspondientes, suma todos esos productos y añade un término de sesgo o bias:

```
salida[o] = input[0] · weight[o][0] + input[1] · weight[o][1] + ...  
           + input[n-1] · weight[o][n-1] + bias[o]
```

Por tanto, la operación básica de una neurona es un producto escalar seguido de una suma de sesgo y una saturación del resultado.

En esta práctica no se utilizará coma flotante. Todos los valores se representarán en formato Q12, es decir, como enteros de 16 bits con signo que representan valores reales multiplicados por $2^{12} = 4096$.

Por ejemplo:

1.0	se representa como	4096
0.5	se representa como	2048
-0.25	se representa como	-1024
-1.0	se representa como	-4096

Cuando se multiplican dos valores en Q12, el resultado queda escalado por 2^{24} , por lo que se dice que el producto queda en Q24:

$Q12 \times Q12 = Q24$

Por este motivo, los productos se acumulan en una variable de 32 bits. Al finalizar la suma, el resultado debe desplazarse 12 bits a la derecha para volver a formato Q12:

```
acc += input[i] * weights[i];    // Q12 × Q12 -> Q24
result = acc >> 12;              // Q24 -> Q12
```

El término bias también está en Q12. Como se suma a un acumulador en Q24, antes de sumarlo debe desplazarse 12 bits a la izquierda:

```
acc = bias << 12;
```

La función principal que calcula una neurona es:

```
int16_t neuron_q12_C(
    const int16_t *input,
    const int16_t *weights,
    uint16_t n,
    int16_t bias_q12,
    int16_t clamp_min,
    int16_t clamp_max
);
```

donde:

- `input` apunta al vector de entrada de la capa.
- `weights` apunta al vector de pesos de la neurona que se está calculando.
- `n` indica el número de entradas de la neurona.
- `bias_q12` es el sesgo de la neurona, también en formato Q12.
- `clamp_min` y `clamp_max` definen el rango permitido para la salida.

Después de calcular la suma, el resultado se satura al intervalo `[clamp_min, clamp_max]`. Esta saturación permite limitar el rango de salida de la neurona y puede utilizarse, por ejemplo, para implementar una activación de tipo ReLU limitada si se fija `clamp_min = 0`.

La función que calcula la capa completa es:

```
uint32_t dense_layer_q12_C(
    const int16_t *input,
    const int16_t *weights,
    const int16_t *bias,
    int16_t *output,
    uint16_t input_size,
    uint16_t output_size,
    int16_t clamp_min,
    int16_t clamp_max
);
```

Esta función recorre todas las neuronas de salida. Para cada neurona llama a `neuron_q12_C`, guarda el resultado en el vector `output` y actualiza un valor de comprobación o *checksum*. El *checksum* se utiliza para facilitar la comparación automática entre las distintas implementaciones.

Los pesos de la capa se almacenan en memoria de forma consecutiva, por filas. Es decir, primero están todos los pesos de la neurona 0, después todos los pesos de la neurona 1, y así sucesivamente:

```
weights = [  
    pesos de la neurona 0,  
    pesos de la neurona 1,  
    pesos de la neurona 2,  
    ...  
]
```

Por tanto, los pesos de la neurona o empiezan en:

```
&weights[o * input_size]
```

Debéis implementar una función envolvente que recorra las neuronas de salida y llame a la función que calcula una neurona individual. Todas las funciones deben cumplir el ABI de la arquitectura ARM utilizado en la práctica, es decir, el paso de parámetros debe realizarse mediante los registros y la pila según el estándar ATPCS/AAPCS.

Esta práctica está diseñada para que algunas funciones tengan más de cuatro parámetros. Por tanto, no todos los parámetros caben en los registros $r0-r3$, y será necesario acceder a la pila para recuperar los argumentos restantes.

En particular, para la función de una neurona:

```
int16_t neuron_q12_C(  
    const int16_t *input,  
    const int16_t *weights,  
    uint16_t n,  
    int16_t bias_q12,  
    int16_t clamp_min,  
    int16_t clamp_max  
);
```

el paso de parámetros será:

```
r0      input  
r1      weights  
r2      n  
r3      bias_q12  
[sp + 0] clamp_min  
[sp + 4] clamp_max
```

Para la función que calcula la capa completa:

```
uint32_t dense_layer_q12_C(  
    const int16_t *input,  
    const int16_t *weights,  
    const int16_t *bias,  
    int16_t *output,  
    uint16_t input_size,  
    uint16_t output_size,  
    int16_t clamp_min,  
    int16_t clamp_max  
);
```

el paso de parámetros será:

```
r0      input
r1      weights
r2      bias
r3      output
[sp + 0] input_size
[sp + 4] output_size
[sp + 8] clamp_min
[sp + 12] clamp_max
```

Debéis tener en cuenta que estos desplazamientos respecto a `sp` corresponden al momento de entrada a la función. Si al comenzar la función se guardan registros en la pila mediante instrucciones como `PUSH`, los desplazamientos necesarios para acceder a los parámetros de pila cambiarán. Este aspecto debe analizarse y documentarse en la memoria.

Requisitos adicionales obligatorios:

- La versión C servirá como referencia funcional. Las versiones en ensamblador ARM y Thumb deberán producir resultados equivalentes.
- Se verificará la equivalencia funcional comparando tanto el valor de retorno como el vector de salida generado por cada versión.
- Debéis medir y comparar el rendimiento y el tamaño de código de las distintas implementaciones: C sin optimizar, C con diferentes niveles de optimización, ensamblador ARM y ensamblador Thumb.
- Podéis crear funciones auxiliares si lo consideráis conveniente, pero las funciones entregadas deberán respetar las firmas indicadas y cumplir el ABI de la arquitectura.

TAREAS A REALIZAR

Paso 1: Estudiar la documentación.

Paso 2: Estudiar, desarrollar y depurar el código inicial del programa.

- Entender las funciones `neuron_q12_C` y posteriormente `dense_layer_q12_C`.
- Debéis monitorizar la ejecución de las **funciones en C, verificando la correcta ejecución**.
- Observad el código en ensamblador generado por el compilador y depurarlo paso a paso tanto en alto nivel como a nivel ensamblador (ASM y THUMB), prestar atención a los cambios en memoria y en los registros del procesador.
- Dibujad el mapa de memoria (direcciones aproximadas de la zona de código, de las variables globales, de las locales en la pila, etc).
- e)** Dibujad el marco de activación de las funciones en pila y el paso de parámetros de las llamadas a funciones que está empleando el compilador.

En la memoria comparar este marco con el que conocéis de AOC1 y con el que os hemos dado en las transparencias.

El mapa de memoria y marco de activación debe entregarse en las entregas e incluirse en la memoria.

Paso 3: Analizar el rendimiento. Hacer uso del analizador de rendimiento (**profiling**) del Keil para analizar el árbol de ejecución y los tiempos de ejecución de las diferentes funciones. Determinar las funciones principales del y las de mayor peso en el tiempo de cómputo.

En la memoria deberéis incluir unas medidas de rendimiento de los distintos códigos, comparar las optimizaciones con la versión en C.

Optimizaciones del compilador. Por defecto el compilador de C (-00) genera un código seguro y fácilmente depurable (*debug*¹) facilitando la ejecución paso a paso en alto nivel. Esta primera versión es buena para depurar, pero es muy ineficiente. Los compiladores disponen de diversas opciones de compilación o *flags* que permiten configurar la generación de código. En concreto hay un conjunto de opciones que permiten aplicar heurísticas de optimización de código buscando mejorar la velocidad o el tamaño. Estas configuraciones se suelen especificar con los *flags* -00 (por defecto, sin optimizar), -01, -02 (código en producción) y -03. La opción -Otime, Optimize for time en los menús de Keil, permite realizar optimizaciones adicionales.

Revisar el código generado por el compilador con niveles de optimización más altos puede ayudarte a generar código en ARM más optimo.

Paso 4: Implementar la función dense_layer_q12_ARM_C en ensamblador ARM. Esta función debe ser equivalente a la versión en C, y llamará a la función neuron_q12_C cumpliendo el ATPCS.

Paso 5: Implementar la función dense_layer_q12_ARM en ensamblador ARM. Debéis tratar de optimizar el código para acelerar el tiempo de ejecución.

El código ARM debe tener la misma estructura que el código C: cada función, cada variable o cada condición que exista en el código C debe poder identificarse con facilidad en la versión de ensamblador. Para ello se deberán incluir los comentarios oportunos.

Paso 6: Implementar la función dense_layer_q12_THB en Thumb. Debéis tratar de optimizar el código intentando reducir su tamaño en bytes.

IMPORTANTE: ES OBLIGATORIO CUMPLIR EL ABI DE ARM EL ATPCS EN LAS LLAMADAS A FUNCIÓN.

Cuando hagáis la llamada en ensamblador debe ser una llamada convencional a una función pasando los parámetros a través de los registros

¹ Te sorprendería saber la cantidad de software comercial generado sin optimizar y en modo debug.

correspondientes o la pila según el estándar **ATPCS**. No sirve hacer un salto sin pasar parámetros, ni pasarlos de una forma distinta al estándar.

El marco de pila (o bloque de activación) de las distintas combinaciones debe ser similar al creado por el compilador para que la comparación sea justa, pero añadiendo lo necesario para simplificaros la vida. **La descripción del marco de pila utilizado deberá aparecer convenientemente explicado en la memoria.**

En todos los casos se debe garantizar que una función no altere el contenido de los registros privados de otras funciones para que no aparezcan errores y bugs indeseados.

Paso 7: Verificación automática y comparación de resultados. Los procesos de verificación y optimización son una parte fundamental del desarrollo del software y deben tenerse en cuenta desde el principio del tiempo de vida del software.

Tenemos diversas implementaciones de las funciones (C-C, ARM-C y ARM y Thumb). Como durante la fase de desarrollo del código en ARM o THUMB es muy fácil que se cometa algún error **no** fácilmente **detectable al depurar** a mano. Vuestro código debe **verificar que las distintas combinaciones generan la misma salida**. Este proceso lo tendréis que realizar múltiples veces, por lo que **se debe automatizar**.

Se evaluará en la entrega la implementación automática del método de verificación.

Para ello, la función **func_verificar** debe realizar esta tarea invocando a las diferentes versiones comprobando que el resultado coincide con la solución esperada. Para simplificar el trabajo vamos a suponer que nos interesa comprobar y comparar el valor de retorno (*checksum*) y el contenido completo del vector de salida.

Con esta primera prueba solo comprobamos un caso. Podéis pensar en crear otros juegos de datos de entrada distintos para verificar más casos. Diseñad un banco de pruebas que compruebe los casos que consideréis relevantes. Incluid una descripción de estos casos en la memoria final.

Paso 8: Medidas de rendimiento. Una vez comprobado que vuestro código funciona bien y que el entorno mide bien los tiempos, vamos a medir los tiempos de ejecución sobre el procesador emulado.

Nos interesa medir **las funciones desarrolladas** (no en todo el programa completo). Debéis comparar el **tamaño del código** en **bytes** de cada versión de las funciones realizadas (C, ARM, Thumb). También queremos **evaluar el tiempo de ejecución** de cada una de estas configuraciones. Calcular las métricas de las combinaciones, comparar y **razonar los resultados obtenidos**.

Debéis incluir un análisis cuantitativo y cualitativo de los resultados obtenidos en la entrega de esta práctica

Paso 9: Estudiar el impacto en el rendimiento del código C de los niveles de optimización (-O0, -O1, -O2, -O3, -optimize for time) y compararlo con las otras versiones en rendimiento, tamaño y si es posible número de instrucciones ejecutadas. Los cambios en el nivel de optimización sólo deberían afectar al rendimiento del código de alto nivel escrito en C y la funcionalidad no debería cambiar. Sin embargo, a veces aparecen errores. Estos errores no son culpa del compilador, son errores que ya estaban en el código pero que no eran visibles. Los errores más típicos son no seguir correctamente el ATPCS, o no asignar **volatile** a una variable que cambia externamente (esto os pasará más adelante). La opción -Otime, Optimize for time en los menús de Keil, permite realizar optimizaciones adicionales que pueden incrementar el tamaño de los binarios como loop unrolling o inlining. Para ver estos efectos debéis por favor activar "Optimize for time" al activar el nivel de optimización -O2 y -O3.

Todos estos resultados deben estar disponibles **el día de la corrección** en una breve infografía, máximo dos folios, y posteriormente quedar claramente reflejados y comentados **en la memoria**.

Nota: que vuestro código en ARM no sea mejor que el generado por el compilador en -O2 o -O3 no es un fracaso. Los compiladores son muy buenos optimizando código. El objetivo no es tanto que ganéis al compilador, como que conozcáis estos flags de optimización y los uséis en el futuro.

PUNTOS DE VERIFICACIÓN EN CADA ENTREGA

ENTREGA 1: INICIO DE LA SESIÓN 2 (SEMANA 3/4 DEL CALENDARIO EINA)

Antes del inicio de la segunda sesión debéis haber realizado correctamente y verificado los pasos 1, 2 y 3. Así mismo debéis haber iniciado los pasos 4 y 5 realizando y mostrar el código en ensamblador para depurarlo y ejecutarlo en la sesión, aunque estén sin acabar de depurar u optimizar. Se verificará:

- El funcionamiento del código en C: **dense_layer_q12_C y neuron_q12_C**.
- El rendimiento de la ejecución.
- El código en ARM de las funciones **dense_layer_q12_ARM_C, dense_layer_q12_ARM y dense_layer_q12_THB**.
- Que entendéis el interfaz binario **ATPCS** utilizado por el compilador y vuestro código ARM (uso de registros, paso de parámetros y resultado), se mostrará al profesor el marco de pila utilizado (en papel o digital).

ENTREGA 2: INICIO DE LA SESION 4

Se deben mostrar

- Las tres principales optimizaciones implementadas en la versión ARM y THUMB.

- Análisis del rendimiento: Los tiempos de ejecución de las distintas versiones de las funciones y niveles de optimización del compilador, así como el tamaño de código. Formato tabla en papel o digital.
- Justificación de la verificación automática implementada.

APARTADO OPCIONAL 1:

Hemos estudiado el impacto en el rendimiento del código C de las opciones compilación con optimización (-O1, -O2, ...). La gran ventaja de que sea el compilador el que aplique las optimizaciones es que nos permite mantener claridad y portabilidad en el código fuente, mientras obtenemos rendimiento en el ejecutable. Si hay algún cambio, lo único que deberemos hacer es recompilar **para sacar de nuevo partido**.

Conforme el compilador aplica las heurísticas el código se hace menos claro. Mira el manual y el código en ensamblador generado por el compilador. Explica qué técnicas ha aplicado en cada versión y como las ha empleado.

Busca optimizar tus funciones ARM aplicando las mismas técnicas. ¿Eres capaz de mejorar el rendimiento obtenido por el compilador?

En algunos casos, nosotros tenemos información sobre el algoritmo o la implementación que el compilador desconoce o debe ser conservador. ¿Eres capaz de optimizar las funciones en C para ayudar al compilador a generar "mejor" código?

EVALUACIÓN DE LA PRÁCTICA

La práctica se presentará (entrega 2) en la sesión 4 de prácticas (2, 6 y 7 de octubre) y se entregará el código en Moodle. La memoria entregará antes del 15 de octubre en Moodle (máximo 5 páginas, fuente 12 o más).

En la presentación se entregarán los códigos fuentes y se comprobará que funciona, que al compilar no aparecen *warnings* y que el código es correcto, así como que el trabajo presentado cumple los requisitos de este documento. Y se deberán mostrar los resultados del rendimiento.

Las fechas y horarios definitivos se publicarán en la página web de la asignatura (moodle).

ANEXO 1: RECOMENDACIONES PARA LA REALIZACIÓN DE LA MEMORIA

La memoria de la práctica tiene diseño libre, la estructura de la memoria se define en el documento **Ayuda_elaboracion_memoria_tecnica.pdf** disponible en la página web de la asignatura (moodle), y es obligatorio que incluya los siguientes contenidos:

1. Portada
 - a. Firma y autoría.
2. Resumen ejecutivo (**una cara máximo**).
 - a. Identificación del problema, Análisis, Alternativas, Implementación, Evaluación, Conclusiones
3. Problema y análisis de rendimiento (*profiling inicial*)
4. Descripción de las optimizaciones realizadas al código ensamblador
 - a. Marcos de activación
5. Resultados de la comparación entre distintas versiones
 - a. Marco experimental y **Análisis** de los resultados
6. Descripción de los problemas encontrados en la realización de la práctica y sus soluciones
7. Conclusiones. Este apartado os suele costar. El objetivo es remarcar aquellos mensajes que queráis que se queden en la cabeza del lector. Pensad que es lo último que va a leer. Algunos temas de los que podéis hablar son: ¿para qué ha servido vuestro trabajo?, ¿qué habéis conseguido?, ¿qué opináis de los resultados?, ¿y de la práctica que os hemos pedido?
8. Anexo: Código fuente comentado para la revisión del sistema anticopia. Además, las **funciones** deben incluir una **cabecera** en la que se explique cómo funciona, qué **parámetros** recibe, dónde los recibe y qué **uso** se da a cada **registro** (ej. en el registro 4 se guarda el puntero a la primera matriz)

Se valorará que el texto sea **claro y conciso**. Cuánto más fácil sea entender el funcionamiento del código y vuestro trabajo, mejor. Una memoria razonable no debe superar las 10 páginas (se puede ampliar con anexos código y figuras)

ANEXO 2: ENTREGA DEL CÓDIGO Y DE LA MEMORIA

La entrega de la memoria será a través de la plataforma docente de la asignatura (moodle en <http://add.unizar.es>). Debéis enviar dos ficheros, el

primero con la memoria y anexos en formato pdf y el segundo un fichero comprimido en formato ZIP con el proyecto completo.

El fichero se nombrará de la siguiente manera que se reconozca la autoría, algo como:

p1_NIP-Apellidos_Estudiante1_NIP-Apellidos_Estudiante2.zip

Por ejemplo: p1_345456-Gracia_Esteban_45632-Arribas_Murillo.zip

Los ficheros independientes deben también identificar los nombres de la pareja, por ejemplo: Memoria_p1_ NIP-Apellidos_Estudiante1_NIP-Apellidos_Estudiante2.pdf para el archivo de la memoria.

No se evaluarán entregas donde no figure ese formato.

ANEXO 3: ALGUNOS CONSEJOS DE PROGRAMACIÓN EFICIENTE PARA OPTIMIZAR EL CÓDIGO ENSAMBLADOR

A la hora de evaluar este trabajo se valorará que cada estudiante haya sido capaz de optimizar el código ARM. **Cuanto más rápido** y pequeño sea el código (y menos instrucciones se ejecuten) **mejor será la valoración de la práctica**. También se valorará que se reduzcan los accesos a memoria.

Algunas recomendaciones útiles:

- No comencéis a escribir el código en ensamblador hasta tener claro cómo va a funcionar. Si diseñáis bien el código a priori, el número de instrucciones será mucho menor que si lo vais escribiendo sobre la marcha.
- Optimizar el uso de los registros. Las instrucciones del ARM trabajan principalmente con registros. Para operar con un dato de memoria debemos cargarlo en un registro, operar y, por último, y sólo si es necesario, volverlo a guardar en memoria. Mantener en los registros algunas variables que se están utilizando frecuentemente permite ahorrar varias instrucciones de lectura y escritura en memoria. Cuando comencéis a pasar del código C original a ensamblador debéis decidir qué variables se van a guardar en registros, tratando de minimizar las transferencias de datos con memoria.
- Utilizar instrucciones de transferencia de datos múltiples como LDMIA, STMIA, PUSH o POP que permiten que una única instrucción mueva varios datos entre la memoria y los registros.
- Utilizar instrucciones con ejecución condicional, también llamadas instrucciones predicadas. En el repertorio ARM gran parte de las instrucciones pueden predicarse. Por ejemplo, el siguiente código:

```
if (a == 2) { b++ }  
  
else { b = b - 2 }
```

Con instrucciones predicadas sería:

```
CMP    r0, #2          #compara con 2
ADDEQ  r1, r1, #1       #suma si r0 es 2
SUBNE  r1, r1, #2       #resta si r0 no es 2
```

Mientras que sin predicados sería:

```
CMP    r0, #2          #compara con 2
BNE    resta           # si r0 no es 2 saltamos a la resta
ADD    r1, r1, #1       #suma 1
B      cont            #continuamos la ejecución sin restar
Resta: SUBNE  r1, r1, #2  #resta 2
Cont:
```

- Utilizar instrucciones que realicen más de una operación. Por ejemplo la instrucción MLA r2,r3,r4,r5 realiza la siguiente operación: $r2 = r3 * r4 + r5$.
- Utilizar las opciones de desplazamiento para multiplicar. Las operaciones de multiplicación son más lentas (introducen varios ciclos de retardo). Para multiplicar/dividir por una potencia de dos basta con realizar un desplazamiento que además puede ir integrado en otra instrucción. Por ejemplo:
 - $A = B + 2 * C$ puede hacerse sencillamente con ADD R1, R2, R3, LSL #1
 - $A = B + 10 * C$ puede hacerse sencillamente con ADD R1, R2, R3, LSL #3 y ADD R1, R1, R3, LSL #1 ($A = B + 8 * C + 2 * C$)

Sacar partido de los modos de direccionamiento registro base + offset en los cálculos de la dirección en las instrucciones load/store. Por ejemplo, para acceder a A[4] podemos hacer LDR R1, [R2, #4] (si es un array de elementos de 8 bits) o LDR R1, [R2, #16] (si es un array de elementos de 32 bits). Además, en ARM se puede desplazar uno de los operandos de la dirección para hacer multiplicaciones de potencias de 2.

NOTA: como el código a desarrollar es pequeño es probable que alguna de estas optimizaciones no sea aplicable a vuestro código, pero muchas sí que lo serán y debéis utilizarlas.

Este guion se ha acabado de elaborar en septiembre de 2026.